Laboratory Work 2 Report

Heorhii Lytvynenko

1 Task Description

This laboratory work analyzes data describing the operations of a parcel delivery and logistics company. The dataset contains semi-structured records of driver stops, including fields such as “marsrutas”, “sustojimo data”, “siuntu skaicius”, “svoris”, “svorio grupe”, “geografine zona”, and others.

1.1 Detailed Task Description

Aggregate the following data into a tabular format, grouped by the ”marsrutas” field: (a) ”svoris” – apply the sum aggregation. (b) ”siuntu skaicius” – apply the sum aggregation. (c) ”geografine zona” – pivot this field by creating a new column for each distinct value, and populate the corresponding rows with their occurrence frequencies.

The task must be solved using Apache Spark with RDD syntax.

Assigned Variant 4

2 Apache Spark (RDD) Approach

Key:

$$marsrutas$$

Value structure:

$$(svoris_sum,\ siuntu_skaicius_sum,\ zonos_map)$$

where $zonos_map$ stores zone frequencies, for example:

$$\{Z1 : 3,\ Z3 : 1\}$$

Processing logic:

- read text file as an RDD;
- split each line into stop records;
- map each stop into (*marsrutas*, partial aggregate);
- reduce by key to merge sums and zone counters;
- collect all distinct zones and generate a pivot table.

3 Pseudocode

3.1 Mapper

```
for each stop record:
    extract marsrutas
    extract svoris (float, default 0)
    extract siuntu_skaicius (int, default 0)
    extract geografine_zona

    zonos = {}
    if geografine_zona exists:
        zonos[geografine_zona] = 1

    emit (marsrutas, (svoris, siuntu_skaicius, zonos))
```

3.2 Reducer

```
for each marsrutas key:
    svoris_sum = sum(partial.svoris)
    siuntu_sum = sum(partial.siuntu_skaicius)
    zonos_map = merge and sum counts by zone

    output (marsrutas, (svoris_sum, siuntu_sum, zonos_map))
```

3.3 Pivot Generation

```
zones = distinct all geografine_zona values
for each marsrutas row:
    create columns Z1, Z2, ...
    fill with counts from zonos_map (or 0)
```

4 Implementation

The implementation is split into separate files:

- `map.py` – parsing each stop and creating initial key-value tuples;
- `red.py` – reducer logic to merge partial aggregates;
- `Sort.py` – sorting routes and collecting all zone columns;
- `main.py` – Spark pipeline orchestration.

4.1 Core Spark Pipeline

```
rawdata = spark_context.textFile(input_path)
parsed = build_parsed(rawdata)
result = parsed.reduceByKey(merge_values)
visos_zonos, rows = sort_result(parsed, result)
print_result(rows, visos_zonos)
```

5 Execution

The job can be run with:

```
spark-submit main.py
```

In this project, the input path is configured as:

```
INPUT_PATH = "duom_cut.txt"
```

6 Results

Using the provided `duom_cut.txt` dataset, the following aggregated table was obtained:

Table 1: Aggregation by route with zone pivot

marsrutas	svoris_sum	siuntu_skaicius_sum	Z1	Z2	Z3
32	7515.74	2936	1210	0	8
33	9131.2	2487	1121	0	2
34	6669.75	1719	689	0	2
35	5888.85	1643	1051	0	3
36	6027.67	1917	402	0	38
37	21489.9	707	263	0	21
38	9595.88	2297	168	0	1014
39	4169.4	988	386	0	2
...					
973	49783.8	1702	228	50	99
974	7838	1072	187	49	528
975	3.6	1	1	0	0
976	5691.78	1622	784	0	197
999	42.6	10	6	0	0

7 Conclusion

Variant 4 was successfully implemented using Apache Spark RDD operations.

The solution correctly:

- grouped records by *marsrutas*;

- computed total parcel weight (*svoris*);
- computed total number of parcels (*siuntu skaicius*);
- generated a pivot table of zone frequencies (*geografine zona*).

The resulting aggregation provides a compact per-route summary that can be used for logistics analysis and route performance evaluation.

8 Repository

The full implementation of this laboratory work is available in the GitHub repository:

- Repository: bdp2 GitHub project
- Colab: BDP2